

Enter hybrid approach

Hybrid approach(Reduced xml, reduced java code + majority annotations)

Steps in Spring programming using annotations

Remove all <bean> tags from the xml config file

0. To enable annotation support -- add context namespace & add the following

<context:annotation-config/> --- To tell SC --to enable class internal annotation support(eg --- AutoWired,PostConstruct,Predestroy,.....)

0.5 --- How to specify location(base pkg) of spring beans to SC?

<context:component-scan base-package="comma separated list of pkgs"/>---

To specify base package/s for scanning spring beans

SC starts searching(scanning) in specified pkgs (including sub-pkgs) ---for classes anno with stereotype annotations --- @Component,@Service,@Repository,@Controller,@RestController,@ControllerAdvice...

Basic class level annotations meant for SC

Super type

@Component --- spring managed bean class , w/o any specific logic

sub - type annotations

@Controller --- In Web MVC scenario -- for request handling bean

@Service --- Service layer (B.L) bean

@Repository --- DAO layer bean

@RestController -- RESTful web service provider bean

@ControllerAdvice -- global exc handler for all controllers

@RestControllerAdvice -- -- global exc handler for all rest controllers

1. @Component --- <bean id class> --- SC interprets it & starts bean life-cycle.

eg ---

package beans;

@Component("abc")

public class MyBean {...}

xml --- <bean id="abc" class="beans.MyBean"/>

OR

@Component

public class MyBean {...}

xml --- <bean id="myBean" class="beans.MyBean"/>

2. @Controller -- spring web MVC req handling controller

3. @Repository --- DAO layer class

4. @Service --- for service layer beans(B.L) .

5. @Scope(value="singleton|prototype|request|session")--- class level annotation --- in xml via scope attribute.

6. @Lazy(true|false) ----class level anno -- lazy-init attribute

7. @PostConstruct ---method level anno - init-method ---method level

8. @PreDestroy ---method level anno-- destroy-method --- method level

9. @Required(true|false) --- setter method or parameterized constr ---tells SC if dependency is mandatory or optional-- def=true

10. @AutoWired ---setter method or parameterized constr or any method or field level
eg --- TestTransport implements Transport {...}
autowire="byType"
eg -- field level annotation ---in ATMImpl bean (dependent)
@AutoWired(required=true) //autowire=byType , mandatory by default (required=true)
private Transport myTransport;
Meaning -- no parameterised constr, no setter , no xml containing bean definition is required.
SC --- chks for any bean of Transport by type & injects it in ATMImpl
What if : SC comes across multiple matches : SC throws NoUniqueBeanDefinitionException
What if : SC doesn't find even a single match : SC throws NoBeanDefFoundException

11. @AutoWired
@Qualifier("dependency bean id")
private Transport myTransport; ---- =>autowire="byName"(w/o a setter!) , field level DI
---spring supplied anno.
SC searches for a bean with id="test"

Single Match found : NO excs , field level D.I succeeds!
Match not found : SC throws exception (UnsatisfiedDependencyException : NoBeanDefFoundExc)
Multiple Matches Found : Ambiguity => SC throws NoUniqueBeanDefinitionException

OR
@Resource(name="dependency bean id")
private Transport myTransport; ---- autowire="byName" , using Field level DI

--J2EE supplied via javax.annotation

SpEL --- spring expression language
dynamic expression language ---spring(3.x) supplied -- to evaluate expressions dynamically.
#{SpEL expression} --- similar to JSP EL --- SpEL allows --- getters, setters, constr invocation, static & non-static method invocations.